Auditable Search for Agentic Quant Research

Draft white paper, revision 5

Kevin Lei

October 19, 2026

Abstract

Frontier LLM agents can search large hypothesis spaces cheaply, and in mathematics that search yields results because a formal checker rejects every candidate that does not follow from the stated definitions. This paper proposes a design for retail quantitative research that borrows the structure but not the certainty: agents generate trading hypotheses freely, and a small controlled kernel enforces specific program-level invariants and keeps a complete, auditable record of what every agent saw. A typed strategy language and a controlled evaluator enforce constraints on data availability, order timing, and unit consistency; an experiment ledger records every evaluation and every result disclosed to an agent; and the whole adaptive search procedure is tested on simulated markets with no edge before any real result is trusted.

We argue that the binding constraint on honest search is the number of independent observations, not ideas or compute, and set the scope accordingly. The design has two research tracks sharing one ledger. A backtestable track searches for systematic strategies across many markets, including the stocks, indices, and commodities now listed as builder-deployed (HIP-3) markets on Hyperliquid, using the long histories of the underlying assets rather than the venue's short one. A forward-only track has agents register event-driven macro theses before their outcomes are known, because an LLM's judgment can be evaluated reliably only on periods after its training cutoff. In both tracks, inference uses only data that played no part in selection: search-period statistics choose candidates, and an untouched later period judges them. Data sources are typed declarations that agents can write strategies against before any feed exists; a feed binds to a declaration only after certification. Reference kernels in Lean 4 and Bend 2 will attempt to prove the language's core invariants, and the two are compared on pre-registered criteria. Formal methods can make certain errors impossible; they cannot turn noisy historical evidence into a correctness certificate, so whether the system controls false discoveries remains a claim to be demonstrated, not assumed.

1 Motivation

Agentic mathematics works as a search problem with a learned heuristic. Many subagents propose candidate lemmas, proof steps, or constructions at high throughput. The LLM acts like the heuristic in classical tree search (GOFAI-style best-first search, or MCTS in AlphaGo-like systems): it decides which branches look promising, not which are correct.

Correctness comes from somewhere else. A proof assistant such as Lean has a small trusted kernel that checks whether each candidate proof follows from the formal definitions, cheaply and mechanically. Because wrong guesses cost almost nothing, the heuristic is allowed to be wild. The kernel's guarantee is also narrower than it looks: it certifies that a proof matches the formal statement, not that the formal statement captures the intended question. A trading evaluator has the same weakness in a stronger form, since it can compute a perfectly correct result for an incorrectly modeled market.

The question for retail traders is how much of this structure carries over to strategy research, with agents as tireless research assistants and formal constraints as part of the kernel. Retail cannot match institutional data, latency, or headcount, but agents lower the cost of research labor. Many current agentic trading tools focus on order entry through a chat interface; this paper is about automating and auditing the research loop instead. Whether agents generate better hypotheses than simpler search methods is itself an open empirical question that the roadmap tests directly.

2 The core obstacle: a noisy, exploitable verifier

The analogy to mathematics breaks at the verifier. A proof checker is exact and cannot be fooled. A backtest is a noisy estimate on non-stationary data, and searching harder against it produces better-looking results whether or not any edge exists.

A pass/fail filter does not avoid this. Accepting only strategies above a Sharpe threshold is still selection, and selection over many candidates behaves like optimization. The example below is illustrative: it assumes independent, stationary, roughly Gaussian returns and independent trials, all of which real strategies violate to some degree. Under those assumptions, over T years of data the standard error of an annualized Sharpe estimate is approximately

$$\mathrm{SE}(\hat{S}) \approx \frac{1}{\sqrt{T}}. \tag{1}$$

With 5 years of data, that is $1/\sqrt{5} \approx 0.45$. Suppose the swarm tests 1,000 strategies with zero true edge and keeps any with an estimated Sharpe above 1, a threshold $\sqrt{5} \approx 2.24$ standard errors above zero:

$$P(\hat{S} > 1 \mid S = 0) = 1 - \Phi(\sqrt{5}) \approx 1.27\%. \tag{2}$$

About 13 null strategies pass. Add 10 genuine strategies with true Sharpe 0.5; each passes with probability $1 - \Phi(0.5\sqrt{5}) \approx 13\%$, so roughly 1.3 survive. In expectation, about 90% of accepted strategies are false discoveries, even though each individual test looked reasonable.

More generally, the expected best of N null strategies is the standard error times the expected maximum of N standard normals, which grows like $\sqrt{2 \ln N}$ to leading order:

$$\mathbb{E}\Big[\max_{i \le N} \hat{S}_i\Big] = \mathrm{SE} \cdot \mathbb{E}\Big[\max_{i \le N} Z_i\Big], \qquad \mathbb{E}\Big[\max_{i \le N} Z_i\Big] \approx \sqrt{2 \ln N}. \tag{3}$$

For $N = 1{,}000$, the leading-order formula gives $0.45 \times 3.72 \approx 1.66$. The exact expected maximum of 1,000 standard normals is about 3.24, which gives 1.45. Either way, the luckiest noise looks like a strong strategy. Agents make this worse in three ways:

1. **Hidden trials.** An agent that tweaks a parameter, looks at the result, and tweaks again has run many tests while submitting one.

2. **Unclear effective N.** A thousand near-duplicates are neither 1,000 independent tests nor one.

3. **Non-stationarity.** Even a correctly validated edge only shows the edge existed in the sample period.

The design goal follows: the system must make the complete history of what agents saw and decided auditable, so that the selection procedure can be evaluated as a whole rather than one test at a time.

2.1 The search budget

Combining the two expressions above gives a rough noise ceiling: the annualized Sharpe that the luckiest of N null trials reaches on T years of data,

$$h(N, T) \approx \sqrt{\frac{2 \ln N}{T}}. \tag{4}$$

This is the leading-order form under the idealized assumptions above, so it slightly overstates the ceiling. It is a motivational approximation for sizing the research program, not part of any decision rule; the gate's actual thresholds come from Section 8.

Effective trials N	3 years	10 years	20 years
10	1.24	0.68	0.48
1,000	2.15	1.18	0.83
100,000	2.77	1.52	1.07

Table 1: Approximate noise ceiling $h(N, T)$ in annualized Sharpe units.

For illustration, suppose a real net-of-cost edge has an annualized Sharpe between 0.3 and 1.0. On three years of data, almost any real search drives the ceiling above that range; on twenty years, a substantial search is affordable. Breadth is the second budget. Grinold's fundamental law [15] says that the information ratio of an active strategy is roughly its skill per independent bet times the square root of the number of independent bets per period. A dozen crypto assets that mostly co-move offer few independent bets. A universe spanning equities, indices, commodities, and rates may offer many more, but the number of instruments is not the number of independent bets: that depends on how correlated a strategy's positions and forecasts are, and it is measured from the covariance of strategy returns rather than assumed.

Time lowers the noise ceiling, breadth raises the signal of a real edge, and trials raise the ceiling. A search is affordable only where the first two are large. This is why the scope of this paper extends beyond the crypto pilot to many asset classes with long histories (Section 11), and why open-ended exploration requires a split-sample design that spends its multiplicity budget only at confirmation (Section 10).

3 Architecture: a small trusted kernel

The system splits into an untrusted generator and a trusted kernel, and the only way to obtain a backtest result is through the kernel (Figure 1). The forward-only thesis track (Section 12) shares the same ledger but has no backtest path at all.

Agents propose strategies with mechanism claims; the kernel type-checks, canonicalizes and logs, evaluates, and gates them. Every result shown to an agent, including for rejected candidates, is recorded in the ledger, because it can shape what the agent tries next.

The kernel's source code should be small, but the trusted computing base is larger than the kernel. Its correctness also depends on:

- historical data reconstruction and ingestion, including revisions and timestamps;
- source certificates and the tests that produce them (Section 5);
- the DSL compiler and runtime, and any proof checker used to verify them (Section 6);

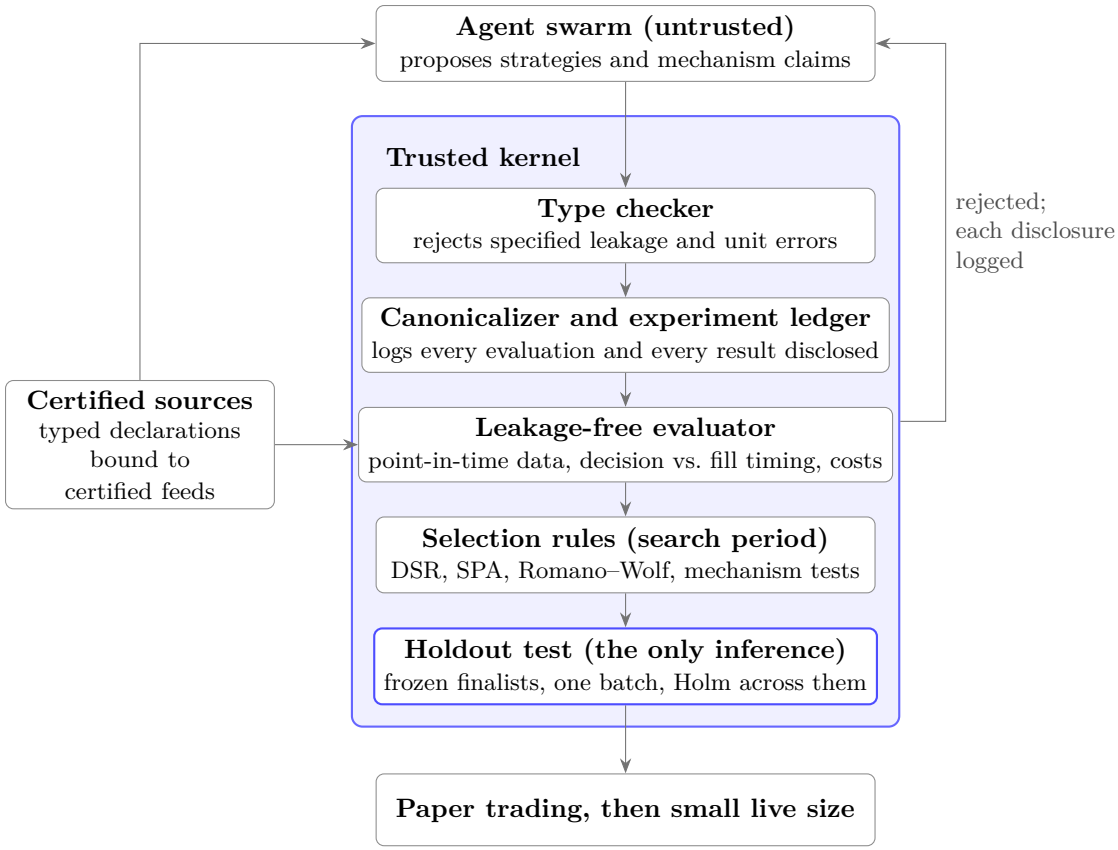


Figure 1: Agents propose freely; results come only through the kernel.

- the market simulator, fee model, and execution model;
- the statistical implementations and the ledger's storage.

Each of these needs stated invariants and tests. "Results come only through the kernel" is a deployment requirement, not a property of the design: agents must run in an environment with no direct access to market data or arbitrary compute over it. An agent that can load a CSV and run its own backtest can do unlogged research, and nothing in the kernel would know.

The deployment therefore needs a stated threat model: the channels through which an agent could learn about data or results, such as files, network access, tools, error messages, caches, other agents, and earlier sessions, and the control that closes each. The pilot runs research agents in an operating-system sandbox whose only tool is a client for a daemon that fixes the data store, enforces the evaluation budget, and discloses a fixed set of fields. Even so, the ledger is an audit mechanism, not a measure of information leakage. It records what was disclosed, not how much an agent inferred from it, and it cannot see what the agent already knew from training.

4 A typed strategy language

Agents do not write arbitrary Python backtests. They write strategies in a small domain-specific language designed to rule out specified classes of research bugs before anything runs. Strategies are data, an abstract syntax tree that the kernel type-checks and interprets, never host-language code. The properties below are design targets; each must be stated as a formal invariant and tested,

including against strategies written specifically to evade it, and the core ones are candidates for proof (Section 6).

Causality in data and in execution. A strategy is a causal stream transformer, formally a Mealy machine: its output at time t depends only on inputs it has received. That alone is not enough. A type signature only describes an interface; the guarantee comes from the runtime feeding the machine only point-in-time data and enforcing four distinct times: when data became available, when the strategy decided, when its order could be submitted, and when that order could actually fill. A strategy that reads a candle's close at 12:00 and fills at that close is causal in its inputs but still leaks through execution, and timestamps alone cannot exclude it, since such a fill can satisfy every time ordering with equality. Even a correct fill rule cannot establish that a fill would really have happened: queue position, cancellations, competing orders, and latency are not fully visible in historical data, so fills are an estimate from a calibrated execution model. Event-driven engines such as QuantConnect's LEAN already separate a time frontier from fill models, and the DSL should at least match that.

```
-- A strategy consumes point-in-time observations and emits orders
type Strategy = Mealy (Obs Event) Order

-- Each observation records when it became knowable and which revision it is
data Obs a = Obs { eventTime :: Time, knownAt :: Time, revision :: Int, value :: a }

-- The simulator, not the strategy, decides submission and fill times
data Fill = Fill { decidedAt :: Time, submittedAt :: Time
                 , filledAt :: Time, price :: Price }
```

Every event therefore carries a sequence number as well as a time, and the evaluator, not these types, enforces two conditions. For every input to a decision,

$$t_{\text{known}} \le t_{\text{decision}} \le t_{\text{submission}} \le t_{\text{fill}}, \tag{5}$$

and the fill is computed only from market events that occur strictly after the order became eligible to execute, ordered by time and then sequence number, so that events sharing a timestamp still have a definite order. Any exception, such as an auction that crosses at a price set after submission, is modeled explicitly.

Consecutive candles share a boundary, which makes this rule easy to misapply (Figure 2). In the pilot, the decision that uses the bar closing at boundary B happens at $B + \delta$, after that bar's data is available. The bar that opens at B is not eligible: its opening price was set before the order existed. The order therefore fills at the open of the following bar, at B plus four hours, the earliest recorded price that provably comes after the decision. The four-hour delay is the cost of having only 4-hour price history; finer history would shorten it.

Units, dimensions, and time bases. Prices, sizes, funding rates, and volatilities carry types, so adding a price to a funding rate fails to compile. Rates also carry their time basis: an hourly and an annualized funding rate are both dimensionless ratios, but combining them without an explicit conversion is a type error. Normalized features get their own dimensionless type. Types do not stop a feature from being normalized with future data; that is the runtime's job.

Levels and changes. Many economic series trend, and two independent trending series routinely show large, meaningless correlations [14]. The type system distinguishes levels from changes.

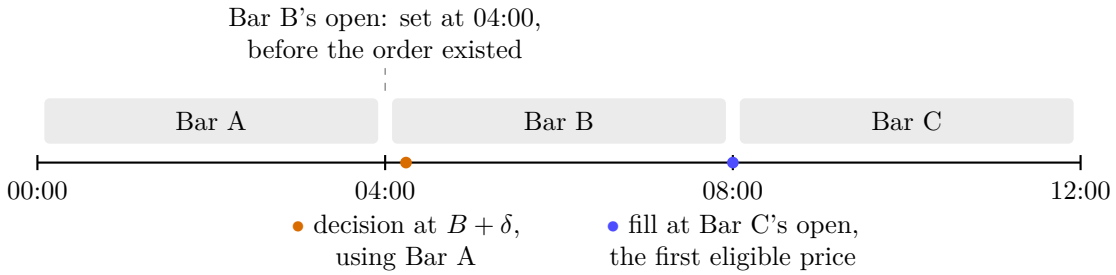


Figure 2: Fill timing in the pilot, with $B = 04{:}00$. Bar A closes and Bar B opens at the same instant, so Bar B's opening price predates the decision and is never used.

Correlation, regression, and ranking operators accept only changes, ratios, or other transforms declared stationary; applying them to levels is a type error unless the strategy explicitly invokes a cointegration test, whose result is a logged statistical finding rather than a license. This enforces declared transformation rules, not stationarity itself: a difference or ratio need not be stationary, and some sound models work on levels. It still removes the most common source of spurious cross-series findings, which matters most for open-ended exploration (Section 10).

Composition. Causal transformers compose: features feed signals, signals feed sizing, sizing feeds risk limits. With objects, morphisms, and parallel composition specified precisely, this should form a symmetric monoidal category, which would give the canonicalizer in Section 7 algebraic laws to rewrite with. That structure is a construction to carry out, not yet a result, and each rewrite law must be shown to preserve full trading semantics, including state, order timing, fees, and floating-point behavior.

Execution realism. Fees, funding payments, slippage, and latency are part of the evaluator's semantics rather than optional parameters. Each venue and market class needs its own model: on Hyperliquid, the fee tier a small account actually pays, funding schedules, and order-book depth at the decision time; for builder-deployed markets, also the market's oracle and trading-hours behavior (Section 11). These models are only as good as their calibration, so the paper treats them as validated components with stated error bounds, not as solved. Where calibration data is thin, for example where historical order books do not exist, the limitation is reported alongside every result that depends on it (Section 8).

5 Sources: declaration, certification, integration

A strategy should be able to say "this depends on a feed of shipping disruptions" before anyone has built that feed. The DSL therefore treats a data source as a typed declaration, separate from any concrete feed.

Declaration. A source declares a schema: an entity key, typed fields with units, a cadence or event semantics, and an availability policy. The `knownAt` time of every observation is assigned by the recorder or the certified backfill, never by the feed itself.

```
source ShippingAlerts : EventStream {
  key      : Region
  kind     : Enum [Closure, Delay, Attack]
```

```
  count    : Count
}  -- knownAt is assigned by the recorder, never by the feed

strategy S (alerts : ShippingAlerts, brent : Price Commodity) = ...

bind ShippingAlerts := feed "alerts-v3"   -- refused without a valid certificate
```

The type checker checks a strategy against the declared schema without any binding. Evaluation requires every source the strategy names to be bound to a certified feed; an unbound source makes evaluation undefined, not empty. Agents can therefore propose hypotheses about data that does not yet exist, and those hypotheses wait in the ledger until the feed is certified. An unbound declaration prevents fitting through this system before the feed binds, but it does not make the strategy a pre-registered prediction: the agent may already know, from training data, published research, or an equivalent feed, how the relationship behaved. Only a frozen strategy evaluated on data independent of its selection supplies independent evidence (Section 8).

Certification. A feed binds to a declaration only with a certificate recorded in the ledger. The certificate holds the results of tests with thresholds fixed before the feed's data is examined against any returns:

- **Availability:** every historical timestamp is classified as directly observed (recorded live, or witnessed by an independent archive capture or a protocol-enforced time), reconstructed from the source's own release records, or estimated. A lag measured on a live recording window validates the availability rule going forward; applied to history it is an assumption, since lags change over time. A historical row's known-at time is therefore the later of the claimed-time rule, used only where the claimed time is protocol-enforced or server-set and the live test passed, and any independent witness. A row with neither is usable only prospectively.
- **Revisions:** a later re-fetch of the same history compared against what was recorded live, so that a backtest never sees corrected values nobody had at the time.
- **Entity mapping:** the precision and recall of the mapping from raw records to schema keys, measured on blind-labelled samples, with thresholds justified by how much mapping error would dilute the signals that use the source.
- **Coverage:** gaps, outages, and the period over which the feed is complete.
- **Extractor validation** for feeds derived by an LLM (below).

Agents may build feeds, but the kernel's certification tests decide whether a feed binds, so an agent never grades its own data. The crypto pilot already follows this pattern for forum and governance sources, where a forum feeds a coin only if its labelled mapping precision is at least 0.8, a threshold fixed before any returns on those inputs existed.

Derived and LLM-extracted sources. A derived source is a DSL function of other sources, available no earlier than the latest of its inputs plus its processing time. When an LLM turns raw text into structured events, it is restricted to factual fields: who, what kind of event, and when. Judgments such as sentiment or importance are excluded, because a model that knows how events turned out can shade those judgments with hindsight. The extractor is validated with names and dates masked, its outputs are cached with the model version and prompt hash, and that version is part of every experiment's identity. Feed design choices, such as which keywords define an event, are

themselves forking paths, so the feed specification is pre-registered before any returns conditioned on it are examined.

Schemas as categories. In Spivak's framework, a database schema is a category and an instance is a set-valued functor on it. Objects are entity types such as Asset, Issuer, Event, and Post; morphisms are functional relationships. Many-to-many, time-varying relationships need their own entities: a wallet does not map to one token, so the schema has a Position object with morphisms to Wallet, Token, and an interval of validity. In these terms a source declaration is a schema, a certified feed is an instance of it, and a strategy written against the schema accepts any certified instance. Ologs are the human-readable version of the same idea.

Migration as functors. Functorial data migration, implemented in the CQL tool [7], moves data between schemas along functors and preserves the declared relationships. It does not solve entity resolution: if two sources use different identifiers for the same asset, or the same identifier for different contracts, a migration can be formally correct and still wrong, which is why entity mapping is a certification test. An open design question is whether this framework earns its complexity over a carefully typed, versioned relational schema; its main advantage would be that migrations and their consistency properties are explicit and checkable.

Point-in-time data. A `knownAt` field is necessary but not sufficient. Real data is revised, backfilled, delayed by ingestion, and sometimes mistimestamped. The store keeps immutable revisions with lineage, so a query "as of time t" returns what was actually available then. Some public archives already work this way: ALFRED [1] serves U.S. macroeconomic series as each vintage was published. Its vintages are dated by day and are typically added within a business day of release, so they establish which value was current on a date, not when it became tradable within that day; intraday release times come from a separate release calendar, certified like any other source.

Cross-source hypotheses. Agents can propose strategies as paths through the schema, for example from a central-bank statement, to the currencies and rate-sensitive sectors it affects, to the commodities those sectors consume. Each such path is a hypothesis recorded in the ledger. A schema with many sources makes it easy to generate many plausible-sounding paths, which is why exploration across sources has its own design (Section 10).

6 Verified reference kernels

The DSL's semantics, type checker, and canonicalization rules are the parts of the kernel whose correctness can be proved rather than only tested. The plan is to build two independent reference kernels, one in Lean 4 [11] and one in Bend 2 [5], prove the same theorems in each, and keep whichever proves more with less risk. Each is differentially tested against the existing engine.

Theorems. Five statements, pre-registered before any proof is attempted:

1. **Non-interference (causality).** For every well-typed strategy s, any two input histories that agree on every observation known by time t produce the same decision at t:

$$h_1|_{\text{known}\le t} = h_2|_{\text{known}\le t} \implies \text{decide}_s(h_1, t) = \text{decide}_s(h_2, t). \tag{6}$$

2. **Unit soundness.** Every well-typed expression evaluates to a value of its declared unit and time basis.

3. **Rewrite soundness.** Each canonicalization rule preserves the strategy's full trading semantics, including state and order timing.

4. **Fill ordering.** Every fill satisfies both timing conditions of Section 4: the time ordering, and a fill price computed only from market events strictly after the order became eligible, in time and sequence order.

5. **Binding.** Evaluation is undefined for any strategy that names an unbound or uncertified source.

Numbers are kept abstract in the proofs. Proving properties of IEEE floating-point code is possible but costly, and several algebraic identities, such as associativity of addition, are false for floats. Rewrite rules are therefore either restricted to float-safe identities, as the pilot's canonicalizer already does, or proved over an abstract ordered field, with the engine's numerical differences bounded by differential tests at a stated tolerance.

What proofs reach. Proofs establish properties of the formal semantics, which is only the first of three levels of assurance (Table 2).

Level	What it establishes	How
Language	Causality, unit soundness, rewrite soundness, binding, and fill ordering in the formal semantics	Proofs (this section)
Implementation	The production engine computes what the formal semantics specify	Differential testing now; a refinement proof, as in verified compilers [23], is future work
Economic model	Simulated fills, costs, margin, and funding approximate the real venue	Empirical calibration only

Table 2: Levels of assurance.

A simulator can satisfy its fill-ordering theorem while charging the wrong fee schedule or misjudging available liquidity, and two implementations that agree can share the same wrong model of the market.

The laws are the new spec. A proof checker certifies that a proof matches its statement, not that the statement says what was intended. If the same agent writes both the law and the proof, it can state a weak law and prove it. The laws are therefore treated like stage criteria: written first, independently reviewed by a model from a different lab, frozen, and never weakened afterwards. A law that turns out to be wrong gets a dated revision, and proofs of the old statement are not counted toward the new one.

Comparison. The two implementations are compared on criteria fixed in advance:

- which of the five theorems are fully proved;

- trust in the checker: Lean 4 is mature, and its proofs can be re-checked by independently implemented kernels. Bend 2 was released in September 2026, describes itself as young and still evolving, and changes quickly, so its exact version is pinned and recorded in every experiment's identity. Bend 2 can also recheck accepted proofs with `-verdict`, which translates them into its core calculus, BendTT, and checks them with a small kernel written and proved in Lean [6]. Its everyday checker and that translation are themselves unproved, and the recheck can run out of fuel on proofs the everyday checker accepts. Judged Bend 2 results therefore require `-verdict`, and a proof accepted only by the everyday checker counts as unproved. Both implementations then rest on Lean at some layer, so their trust bases are not fully independent;
- performance on a fixed replay benchmark and on null-simulation throughput, where Bend 2's advertised automatic CPU and GPU parallelism may help, since thousands of simulated markets are independent; vendor benchmarks are not taken as evidence;
- agent effort to reach each milestone, measured rather than assumed: Bend 2 ships an agent-oriented guide and a laws-and-proofs workflow designed for agents, while Lean 4 has far more published code and proof automation;
- agreement with the existing engine under differential testing.

The winner becomes the reference kernel. If it is also fast enough to replace the engine, the engine is deleted; otherwise the engine stays and is permanently differential-tested against it. The losing implementation is deleted either way. Independence is already supplied by the engine itself and by components implemented separately by a model from another lab, so a second proof kernel kept in lockstep would add maintenance more than assurance.

7 Canonicalization and the experiment ledger

The ledger makes the research process auditable; it does not by itself produce a trial count that can be plugged into a correction. A canonical strategy is a program, while a statistical trial is about information: what an agent saw and how that shaped what it selected next. An agent that inspects one strategy's drawdown, yearly returns, and cost sensitivity, then proposes something new, has spent multiplicity without creating any new canonical form. Conversely, rerunning an identical deterministic backtest, or rejecting a strategy at the type checker before it touches data, spends none.

What the ledger records. Six things, kept distinct:

1. **Strategy identity:** the canonical form, so equivalent programs are recognized.
2. **Experiment identity:** strategy, parameters, asset universe, data window, dataset and certificate versions, availability-time conventions, execution and fee model versions, extractor versions, simulator version, and random seeds.
3. **Disclosures:** every statistic or plot returned to an agent, including for rejected candidates.
4. **Selection decisions:** which candidates an agent or the gate chose to pursue, and what it had seen at that point.
5. **Source certificates:** the tests and thresholds under which each feed was bound.

6. **Theses:** registered forward predictions and their retrieval logs (Section 12).

Multiplicity is then estimated from this history, and those estimates are checked by testing the whole procedure on null markets (Section 16).

Canonical forms. Strategies are rewritten using the DSL's algebraic laws, for example commuting independent features or folding constant transforms. Equality saturation with e-graphs, as in the egg library, is a practical tool for this, with an important limit: it finds expressions provably equal under the supplied rewrite rules and extracts one by a cost model. It does not produce a unique canonical form for every semantically equivalent program, and results depend on rule coverage and resource limits. The claim is therefore narrower: the system detects some provable equivalences, conditional on the soundness of its rewrite rules, which Section 6 aims to prove.

Effective N. Strategies that differ syntactically can still behave almost identically. Clustering return streams by correlation gives one estimate of the effective number of independent trials, in the spirit of Bailey and López de Prado, but it is a heuristic. The correct adjustment depends on the joint distribution of the test statistics, and the correlations are estimated on the same sample being searched. The gate therefore reports how its decisions change across several effective-N estimates, and treats a promotion that holds only under the most favorable estimate as fragile.

Search guidance. Agents can query which regions of strategy space are already explored, steering generation toward novel hypotheses. This is itself a feedback channel: telling agents where past results were good or bad is a disclosure, and it is logged and accounted for like any other.

8 The statistical gate

Individually valid tests do not make a valid combined procedure. The gate therefore separates selection from inference. Everything computed on the search period, including the screen, SPA, Romano–Wolf, and the mechanism check, is a selection rule: it decides which candidates reach the holdout, and its behavior is calibrated by simulation (Section 16), but it supports no error claim on its own. Romano–Wolf's guarantees, for example, do not survive applying it to a family that was itself chosen on the same data. The only inferential step is the holdout batch: finalists frozen without any holdout data are tested once on that untouched period, with Holm's correction across them. Each campaign runs a fixed gate written down before it starts, and its error rate is claimed only for its finalist family on its holdout, under the holdout test's scope assumption that losses are capped at equity.

1. **Search period only.** Agents see results from a research period. The holdout period is never shown to them.

2. **Screen.** Candidates must pass a deflated Sharpe threshold computed at each of several effective-N estimates, using the largest. That is conservative only relative to the estimators chosen, not a guaranteed bound.

3. **Global screen.** Hansen's SPA runs over every strategy evaluated in the campaign and asks whether any beats the benchmark. If it does not reject, the campaign promotes nothing, which under the complete null stops most campaigns before any holdout is spent.

4. **Ranking.** Romano–Wolf stepdown ranks the screened candidates by adjusted p-value, and at most a pre-set number become finalists. These p-values are used for selection only, since the screened family was chosen on the same data.

5. **Mechanism check.** A candidate whose pre-registered mechanism prediction fails is removed from the finalists. There is no override by a higher statistical threshold.

6. **Freeze.** The finalists, the promotion rule, and the multiple-comparison correction across finalists are fixed before the holdout is opened.

7. **Holdout batch.** All finalists are evaluated together, once. A finalist is promoted to paper trading only if its net-of-cost holdout return is positive and its holdout p-value for the test below passes a preset level after Holm correction across finalists. The thresholds are parameters; the structure is fixed.

8. **No recycling.** A strategy modified after its holdout result cannot be retested on that holdout. A holdout used by one campaign is spent: the next campaign evaluates on a later period, and the old holdout becomes part of its search data.

The holdout test. For each finalist i, the null hypothesis is that at every step of the holdout its net return has non-positive mean given everything before that step, which includes everything the search revealed:

$$H_{0,i}: \ \mathbb{E}\big[r_{i,t}^{\text{net}} \mid \mathcal{F}_{t-1}\big] \le 0 \quad \text{for every } t, \qquad \mathcal{F}_{t-1} \supseteq \mathcal{F}_{\text{search}}. \tag{7}$$

This is what having no edge means: a return that can be predicted to be positive at some step is an edge, since a strategy conditioning on the prediction would exploit it. Because finalists are frozen using search data only, each is a fixed strategy given $\mathcal{F}_{\text{search}}$, so the test needs only p-values that are valid for a fixed strategy:

$$P(p_i \le u \mid \mathcal{F}_{\text{search}}, H_{0,i}) \le u \quad \text{for all } u \in [0,1]. \tag{8}$$

No test of a mean is both valid for every finite-mean distribution and powerful [2]: small steady gains with a rare large loss look like an edge whenever the loss is missing from the sample. The gate therefore makes one scope assumption explicit and enforces it. Losses are capped at account equity, as on the venue, so every per-step net return satisfies $r_t \ge -1$; the kernel enforces the cap, and its reference model proves that it holds. Under that bound the gate tests by betting [36]. A bettor starts with wealth 1 and at each step stakes a fraction $\lambda_t \in [0, 1/2]$ of its wealth on the next return, chosen from earlier returns only, so its wealth is $K_t = \prod_{s \le t}(1 + \lambda_s r_s)$. The gate's bet is the plug-in Kelly fraction, the sum of past returns over the sum of their squares, truncated to $[0, 1/2]$. Under $H_{0,i}$, K is a nonnegative supermartingale from 1, so by Ville's inequality [35], $P(\max_t K_t \ge 1/u) \le u$, and

$$p_i = \min\left(1, \ 1/\max_t K_t\right) \tag{9}$$

is valid at every level. That holds for every return process in the null, with any dependence, heteroskedasticity or sparsity, and with no simulation. The test is still run once on a locked suite of null series that was specified independently and implemented by another model. That suite checks the implementation and the bound, not the validity argument. Holm's correction then bounds the probability of any false promotion among the finalists. Validity costs power, and on a holdout of a few months the cost is large. With $n = 546$ four-hour steps, a finalist reaches $p \le 0.05$ only with a mean return near 1% per step. Even an untruncated Kelly bet would need an annualized Sharpe ratio near 5, and any valid test faces the same limit at that length. Zero promotions is therefore the expected outcome, even for a real edge of ordinary size.

How many simulated campaigns. With no false promotions in n independent complete-null campaigns, the exact one-sided 95% upper bound on the promotion probability is

$$1 - 0.05^{1/n}, \tag{10}$$

so 59 campaigns are needed to support a bound below 5%, and 299 to support 1%. Because inference rests on the holdout test, the expensive part of the pipeline, the research agent, does not need to be in most of these runs. Calibration uses hundreds of campaigns driven by cheap baseline searchers, together with a locked check of the holdout test; agent-driven campaigns are smoke tests that report the much weaker bounds they support. With no promotions observed, 20 agent campaigns bound the rate only to about 14%.

Across campaigns. Error control is per campaign. A project that runs many campaigns will promote some noise even if every campaign is calibrated: under the complete null, 20 independent campaigns at 5% each promote at least one false strategy with probability $1 - 0.95^{20} \approx 64\%$. The ledger therefore reports every promotion beside the number of campaigns run. If a claim about the whole project is wanted, it is family-wise: campaign j is tested at level α_j with $\sum_j \alpha_j \le \alpha$, for example $\alpha_j = \alpha\, 2^{-j}$, which bounds the probability of any false promotion across all campaigns. The budget is fixed before the first real campaign, and it makes later campaigns progressively harder to pass. Sequential procedures such as alpha-investing [13] control a different and weaker quantity, the marginal false discovery rate, and are not a substitute for this guarantee. Successive holdouts are later periods, so campaigns are naturally ordered in time.

Reported, not judged. Beside each finalist's judged holdout result, the gate reports its holdout return at twice the modeled costs and the share of the holdout period whose market conditions fall within the range the cost model was calibrated on. These reports appear only after the promotion decision and are never used to choose among finalists. They exist because cost models calibrated on short windows may understate costs exactly when markets move and strategies trade most.

Method	What it controls	Key assumption to check
Deflated Sharpe ratio	Selection bias from N trials, non-normal returns, track length	A valid effective N and the Sharpe sampling model
White's Reality Check, Hansen's SPA	Whether the best of a family beats a benchmark	A correctly specified family and a suitable bootstrap for dependent data
Romano–Wolf stepdown	Family-wise error across correlated strategies	Resampling assumptions; it controls error, it does not certify survivors
Benjamini–Hochberg [8]	False discovery rate	Valid p-values and acceptable dependence
Probability of backtest overfitting (CSCV)	Instability of the in-sample selection procedure	A diagnostic of the procedure, not a forecast of future profit

Table 3: Statistical methods used by or available to the gate.

The deflated Sharpe ratio compares the observed Sharpe with the Sharpe expected from the best of N null trials, adjusting for skewness, kurtosis, and sample length. Unlike Section 2, it works in per-period units:

$$\mathrm{DSR} = \Phi\left(\frac{(\hat{S} - S_0)\sqrt{T-1}}{\sqrt{1 - \hat{\gamma}_3 \hat{S} + \frac{\hat{\gamma}_4 - 1}{4}\hat{S}^2}} \right). \tag{11}$$

Here T is the number of return observations, $\hat{S}$ is the Sharpe per observation period (for daily data, roughly the annualized Sharpe divided by $\sqrt{252}$, or $\sqrt{365}$ for markets that trade every day), and S_0 is the expected maximum per-period Sharpe under the null given N effective trials. $\hat{\gamma}_3$ is skewness and $\hat{\gamma}_4$ is ordinary kurtosis, which equals 3 for a normal distribution, not excess kurtosis. The formula assumes independent, identically distributed observations. Autocorrelation changes the Sharpe ratio's sampling distribution [25], and swapping in a robust variance does not by itself fix the null distribution of the maximum across correlated strategies. The DSR expression is therefore a baseline under its stated assumptions; the gate's actual thresholds are calibrated with time-series resampling, such as a block bootstrap, and with the simulations in Section 16. Folds use walk-forward splits with purging at boundaries [26] so overlapping labels do not leak.

The holdout. Every query to a holdout leaks some information, and what matters is how much each answer reveals, not just how many queries there are. The reusable holdout of Dwork et al. (Thresholdout) releases holdout answers through a noisy threshold mechanism and gives formal guarantees under adaptive queries. If Thresholdout is not implemented, the system's hard query budget is a heuristic safeguard only, and the paper does not claim formal adaptive validity for it.

9 Mechanism obligations

Every strategy must ship with a stated mechanism, and the mechanism must imply tests beyond PnL. These tests are additional empirical checks that can count against a strategy; they are not a second verifier comparable to a proof checker. The obligation has three parts:

1. **Counterparty.** Who is on the other side of the trade, and why are they willing to trade at this price? The counterparty need not be wrong: forced sellers at liquidation, hedgers paying for immediacy, index funds rebalancing on a schedule, and traders reducing inventory can all be acting rationally.

2. **Conditional predictions.** If the mechanism is real, the effect should vary in specified ways, with stated tolerances. A liquidation-cascade strategy should earn more when positions are crowded near liquidation and less when funding has normalized. A slow-information strategy should decay as the information ages.

3. **Capacity and decay.** Roughly how much size the effect supports, and what would make it disappear.

Conditional predictions are written down before they are tested and logged in the ledger like any other experiment. Ideally they are tested on data not used to find the strategy, since predictions checked on the same sample are not independent of the PnL result. A failed prediction is evidence that the stated explanation is wrong, not proof that the strategy is: it may work for another reason, or the test may lack power. Under the gate in Section 8, a failed pre-registered prediction removes the candidate from that campaign's finalists; it can be re-proposed with a revised mechanism in a later campaign, evaluated on fresh data.

A working hypothesis of this design is that LLMs are most useful here for proposing economically plausible ideas rather than for blind search. That hypothesis should be tested against simpler generators, such as random or genetic search over the same DSL, using the same gate.

10 Exploration across domains

For a retail researcher, "alternative data" cannot mean buying proprietary feeds. It can mean using agents to look for relationships between sources that no human would think to combine, for example shipping disruption reports, regional electricity prices, and a basket of industrial equities. This is also the statistically most dangerous activity in the system: the space of cross-source combinations is effectively unbounded, so by Table 1 the best in-sample result of an open-ended exploration is uninformative on its own, however striking it looks.

The design therefore uses the same split as the gate, the classical data-splitting approach [10], with two changes for open-ended search:

1. **Free exploration.** On the search period agents search without any screen. Every evaluation is still logged, but nothing found counts as evidence; a screen at the exploration's N would reject everything.

2. **A small, fixed number of finalists.** Agents promote at most k hypotheses, a number fixed in advance, each with a mechanism and with its parameters, source definitions, and asset selection frozen.

3. **Confirmation on untouched data.** The k finalists are tested once on a later period, separated from the search period by an embargo gap, with Holm's correction across k. The multiplicity paid is k, not the size of the exploration, which is what makes the design affordable. Where history allows, an intermediate confirmation period can precede the final holdout, so that only hypotheses confirmed once reach it.

The qualification is independence. An embargo reduces serial dependence between periods but does not make later markets independent of earlier ones, and any decision made after looking at confirmation data invalidates it. Splitting also costs data: each period is unavailable to the others, and each confirmation period, like a holdout, is spent once used. The design is therefore practical only on long histories, which is a further reason for the multi-asset scope. The level-versus-change types (Section 4) guard against the most common spurious cross-series finding, and the mechanism obligation applies with particular force: a relationship between unrelated domains must name the economic channel that links them, or it does not reach confirmation.

An honest caveat: a relationship nobody would think to trade may persist precisely because nobody trades it, but it is far more likely to be noise. The expected outcome of most exploration campaigns is zero confirmed hypotheses.

11 Multi-asset research and HIP-3 markets

Hyperliquid's HIP-3 standard lets third-party builders deploy perpetual futures markets [20], and builder-deployed markets on equities, equity indices, and commodities have grown quickly. This makes a single venue a plausible execution layer for many asset classes, for operators who may use it. The research layer, however, should not use those markets' own histories, which are months long. It uses the long histories of the underlying assets, and treats each venue market as an execution mapping that must be certified separately.

Research layer. Each asset class needs point-in-time data:

- **Equities:** prices adjusted for splits and dividends, and a universe that includes delisted companies. Freely available equity data usually omits delisted names, which biases every

backtest upward. Survivorship-free sources such as CRSP exist but carry licensing terms that must permit the intended use.

- **Indices:** index levels and, for constituent strategies, point-in-time membership.
- **Commodities:** futures series with explicit, versioned roll rules, since the choice of roll method changes returns.
- **Macro and rates:** vintage-correct releases (Section 5) and a calendar of scheduled events with their release times.

Universe selection is itself a lookahead. The set of assets that have HIP-3 markets today was chosen in 2025 and 2026, largely by popularity, and popularity reflects past performance. A twenty-year backtest restricted to today's listed stocks is a backtest on yesterday's winners. The research universe must therefore be defined point-in-time, for example by index membership as it stood on each date, and the list of currently listed venue markets is used only to map research positions to execution.

Execution mapping. Before a research result on an underlying asset can be traded through a venue market, that market is certified like a source:

- **Tracking:** the gap between the venue's mark price and the underlying, measured over a recording window, including around corporate actions such as dividends and splits, whose handling may differ by deployer.
- **Closed hours:** the perpetual trades continuously while many underlying markets close overnight and at weekends. Price discovery then happens on the venue alone, and reopening gaps against the underlying must be measured, not assumed.
- **Funding and costs:** the funding mechanism and realistic costs for the account size.
- **Market risk:** deployer-supplied oracles and parameters, and the possibility that a builder market is halted or delisted.

A strategy whose edge depends on the closed-hours behavior of a venue market cannot be researched on the underlying at all, because the underlying has no prices in those hours; it falls back to the venue's own short history and the corresponding statistical penalty.

A signal study, not a venue backtest. A backtest on an underlying asset establishes that a signal predicted the underlying's returns. It is not a backtest of the venue market, whose returns also include funding, basis changes against the underlying, mark-price dynamics, liquidation risk, and costs, any of which can change the sign of an edge. Results on underlyings are therefore reported as signal studies. A projection to a venue market applies a certified execution mapping and is labelled model-dependent until the market's own forward record confirms it.

Venue eligibility. Eligibility is part of feasibility, not a final step. Hyperliquid's interface restricts some jurisdictions, including the United States, so an operator in one of them cannot assume access to its markets. A research result on an underlying asset is portable: the execution mapping can target any venue the operator may use, including regulated instruments on the same underlyings, and the certification steps above then apply to that venue instead.

Breadth from a pilot. Stages 0 to 5 of the roadmap build and test the machinery on crypto perpetuals. Extending to these asset classes keeps the architecture: the kernel's role, the gate, and the ledger. The present kernel, however, models only a perpetual that trades around the clock, with mandatory funding and a fixed 4-hour decision grid. Stage 8 therefore generalizes it to instrument kinds with their own calendars, settlement, and decision cadences, alongside new sources, certificates, execution mappings, and null models whose cross-asset dependence matches the larger universe.

12 The thesis track: event-driven macro

The research direction we consider most promising for an agent-assisted retail trader is discretionary, event-driven macro: theses about how a scheduled or unscheduled event, such as a central-bank decision, an inflation release, an earnings report, a supply disruption, or a regulatory decision, will move a specific exposure over days to weeks. This is where reading and synthesis matter most and where speed matters least.

Why this track is forward-only. An LLM's judgment cannot be reliably backtested on a period its training data may cover: a model reading news from 2024 may already know what happened next, even if indirectly. A frozen model can be evaluated on a period that begins after its training cutoff, as the pilot does for its single-agent campaign, but cutoffs are imprecise and later fine-tuning can leak later information, so the margin must be generous and the usable period is short. LLM forecasting research reaches the same conclusion and evaluates on questions resolved after the model's cutoff [16, 22]. Theses are therefore registered before their outcomes are known and evaluated only forward.

Registration. A thesis is a structured record, hashed into the ledger at registration and never edited:

```
thesis T-0142 {
  registered   : set by the kernel at write time
  exposure     : long crude oil vs. short airline basket
  horizon      : resolves at a fixed date, three weeks out
  claim        : P(exposure return > 0 over the horizon) = 0.64
  catalyst     : a scheduled producer meeting; reported inventory draws
  invalidation : close early if the exposure loses more than a stated amount
  sources      : ids in the retrieval log
  size         : fixed risk budget / distance to invalidation
}
```

The retrieval log records every document the agent read through the system and when. It does not capture what the model already knew, which is one more reason the track is forward-only. Sizing is a fixed risk budget: the position is sized so that exiting at the invalidation level would lose a pre-set fraction of equity. Realized losses can exceed that target when prices gap through the level, fills slip in thin markets, or several theses lose together, so total risk across concurrent theses is capped as well. The stated probability is a forecast to be scored, not a sizing input, because probability and volatility alone do not determine a sound position size without payoff magnitudes, costs, and dependence between positions. The agent does not choose position size. Early exits are allowed only through the pre-declared invalidation condition; any other exit is logged as a separate decision, and the thesis is scored both as registered and as managed, so that discretionary exits cannot hide losers.

Scoring. Each thesis is scored two ways. Its stated probability is scored with the Brier score [9],

$$B = \frac{1}{n}\sum_{i=1}^{n}(p_i - o_i)^2, \tag{12}$$

where $o_i \in \{0, 1\}$ is the outcome, and reported as a skill score $1 - B/B_{\text{ref}}$ against baselines: the historical base rate of the same outcome over the same horizon, market-implied probabilities where options or outcome markets provide them, an ablated agent that registers theses without reading any sources, and a frozen benchmark policy that picks exposures and horizons by a simple pre-specified rule. Because the agent chooses its own exposures, spreads, and horizons, a fixed 50% baseline is not appropriate: drift, carry, and costs move the natural base rate, so it is estimated for each chosen exposure. Its paper position is scored by return under the fixed sizing rule. Theses that share a catalyst or theme are clustered, and the effective sample is the number of clusters, not theses.

Sample size. Forward evaluation is honest but slow. Detecting a hit rate p_1 against a base rate p_0 with a two-sided test at the 5% level and 80% power needs roughly

$$n \approx \left(\frac{z_{0.975}\sqrt{p_0(1-p_0)} + z_{0.80}\sqrt{p_1(1-p_1)}}{p_1 - p_0} \right)^2 \tag{13}$$

independent theses: about 194 for $p_1 = 0.60$ against $p_0 = 0.50$, and about 780 for $p_1 = 0.55$. Dependence between theses raises these numbers, and a base rate other than 50% changes them. At a few concurrent multi-week theses, that is several years. Probability scoring extracts more information per thesis than a hit rate, and many smaller, shorter-horizon theses shorten the timeline, but the track cannot produce strong evidence quickly. Because its evidence accrues only with elapsed time, it should start as early as possible.

Connection to the backtestable track. The thesis agent may use the backtestable track's certified sources and promoted strategies as inputs. In the other direction, the registered theses form a point-in-time record whose `knownAt` is the registration time, so once enough have accumulated, the thesis log itself can be certified as a source for the backtestable track.

13 Research directions for retail edge

The method is not the edge: institutions already run automated alpha research. The directions below are hypotheses about where a small account might compete, not demonstrated sources of return.

- **Event-driven macro theses** (Section 12). Large funds run discretionary macro and event-driven books too, but at small size an agent-assisted researcher can cover many minor catalysts that do not justify an institutional analyst's time.
- **Broad, known premia.** Searches for novel technical patterns rarely survive correction for data snooping [32]; the technical effects with long records are broad premia such as time-series momentum [28, 19], which work through diversification across many markets. These are also available cheaply through funds, so a homemade version must beat them after costs.
- **Cross-domain relationships** found through split-sample exploration (Section 10).

- **Small capacity.** Some strategies support too little size to interest a large fund, though institutions do not ignore small opportunities uniformly.
- **Venue structure on Hyperliquid.** Positions and margin for given addresses are far more observable than on centralized venues, which makes crowding, deleveraging, and funding dislocations testable in principle. In practice, liquidation prices depend on cross-margin account state, funding paid, and partial-liquidation rules, so they cannot be rebuilt from entry price and leverage alone, and historical archives can have gaps.
- **Slow, messy information.** Governance proposals, filings, and long-tail discussion may be absorbed by the market slowly; LLMs make reading them cheap, though they make it cheap for competitors too.

Observed wallet activity is treated as data rather than something to copy directly. Among thousands of visible wallets, many strong records are luck, which deflated statistics can partly correct. Survivorship is a different problem: wallets that blew up or were abandoned may be missing from whatever set looks visible today, and no adjustment to the survivors' Sharpe ratios recovers them.

14 Related work

Agentic research automation itself is not new. The possible contribution is narrower: combining a restricted strategy language with proved core invariants, typed and certified sources, a full record of what agents saw, point-in-time evaluation, multiplicity-aware inference validated by null simulation, and a forward-only thesis track in one ledger. Establishing that this combination adds something requires a direct comparison with the systems below, which this draft does not yet make.

- **Automated quant research.** Microsoft's R&D-Agent-Quant [24] already runs a closed loop of hypothesis generation, code development, market evaluation, and feedback. Alpha-GPT [34] uses LLMs to mine alpha factors interactively. Kakushadze's 101 Formulaic Alphas [21] publishes formulaic alphas used in industry.
- **Backtesting infrastructure.** Event-driven engines such as QuantConnect's LEAN [27] already use a time frontier to limit future-data access and separate fee and fill models. The DSL here must offer stronger, stated guarantees to justify itself.
- **Multiple testing in finance.** Harvey, Liu and Zhu [18] argue for a t-statistic hurdle near 3 for new factors. Bailey and López de Prado introduce the deflated Sharpe ratio [4], and with Borwein and Zhu the probability of backtest overfitting [3]. White [37], Hansen [17], and Romano and Wolf [29] provide bootstrap tests for data snooping, and Sullivan, Timmermann and White [32] apply them to technical trading rules.
- **Adaptive data analysis, data splitting, and sequential testing.** Dwork et al. [12] show how to reuse a holdout set under adaptive queries with formal guarantees, for independent samples rather than dependent time series; Cox [10] analyses the classical split-sample design used by the gate; Foster and Stine [13] give alpha-investing for sequences of tests.
- **Forecasting.** Brier [9] introduced the probability score used for theses; Tetlock and Gardner [33] document what distinguishes skilled human forecasters; recent work evaluates LLM forecasters on questions resolved after their training cutoffs [16, 22].

- **Formal methods.** Equality saturation (egg) [38] supports detecting equivalent expressions; Lean 4 [11] and Bend 2 [5] provide the proof checkers for the reference kernels; CompCert [23] shows what a refinement proof connecting formal semantics to an implementation involves.
- **Categorical data integration.** Spivak and collaborators develop ologs [31], functorial data migration [30], and the CQL tool [7].

15 Open problems and risks

The weakest link is non-stationarity: the inference methods in Section 8 rely, in different ways, on the sample being informative about the future, and markets drift and adapt.

- **Multiplicity under adaptivity.** Adaptive-inference theory exists, but no general method turns an agent swarm's interaction history into a trustworthy multiplicity estimate. Null simulation of the whole procedure is the practical check, and inference is confined to untouched holdouts.
- **Overfitting to the null simulator.** A gate refined until it passes its own null models may fail on others. Null models used while designing the gate are kept separate from a locked evaluation suite, specified independently and run only to judge the finished procedure, and calibration is reported under deliberately misspecified execution models too.
- **Error across campaigns.** Per-campaign control does not bound the project's lifetime false-promotion count (Section 8).
- **Incomplete canonicalization.** Behavioral equivalence is undecidable for expressive languages, though decidable for finite-state Mealy machines over finite alphabets. A DSL with real-valued state falls on the hard side. Return-correlation clustering flags behaviorally similar strategies but is only a similarity heuristic.
- **Expressiveness trade-off.** A DSL strict enough to forbid leakage may forbid useful strategies. Extensions are acceptable only if they preserve the stated invariants and the proved theorems.
- **Proof-checker trust.** A soundness bug in a young proof checker would vouch for leaky strategies. This is the main risk of the Bend 2 implementation and the reason checker maturity is a comparison criterion.
- **Hindsight in LLM-derived data.** Masked validation reduces, but does not eliminate, the chance that an extractor's outputs reflect what the model knows about later events.
- **Data quality and licensing.** Availability times for text sources are hard to establish after the fact, archives can be revised or incomplete, and survivorship-free data for traditional assets often carries licensing restrictions.
- **Short venue histories.** Hyperliquid's core markets have a few years of data and its builder-deployed markets only months. Researching on underlying assets mitigates this only for edges that do not depend on venue-specific behavior.
- **Slow forward evidence.** The thesis track needs years to distinguish modest skill from luck (Section 12).

- **Venue and access risk.** Smart-contract, oracle, deployer, and liquidity risks can be partly modeled, but their severe tails are poorly represented in short samples. Live trading must use a venue the operator is permitted to access under its terms and applicable law, which is checked during feasibility (Section 11); the research pipeline itself is venue-agnostic.
- **Crowding.** If the approach spreads, the small-capacity edges it targets may shrink.

16 Roadmap

Check that the data can support the research before building the kernel, and build the kernel before the swarm. Evaluation criteria for each stage are fixed before the data they judge is inspected, so refining the evaluation does not become another layer of selection. Stages 0 to 5 form a pilot on Hyperliquid's core crypto perpetuals; stages 6 to 10 extend it. Stages 6 and 10 depend only on the pilot's kernel and ledger and can run while the pilot's evidence accumulates. Stages 7 to 9 change the kernel, so they begin after the pilot exits stage 5, and every judged pilot result comes from one frozen kernel. Each experiment records the kernel's build hash, so a change made before then would be visible in the ledger.

0. **Feasibility.** Pick one strategy family, for example funding dislocations, and confirm that every feature it needs was observable at each historical decision time and that realistic execution can be simulated. Version the venue's trading and margin rules alongside the data, since they can change over time. Exit when a point-in-time replay of that family is possible, or stop and pick another family.

1. **Kernel MVP.** A minimal DSL with stated invariants, a point-in-time evaluator with fee, funding, and fill models, and a ledger that records evaluations and disclosures. Exit when an adversarial test suite, including strategies written to evade the rules, fails as specified.

2. **Null, mixed, and power calibration.** First define "no edge" for each null model, for example zero expected net-of-cost return for every strategy expressible in the DSL. Preserving raw return autocorrelation contradicts this, since autocorrelation is itself exploitable; models such as martingale-difference processes with volatility clustering and cross-asset dependence keep the volatility structure without a predictable mean. Then run the entire adaptive search and gate on many simulated markets and measure three things against targets fixed in advance:

   - **Complete null:** the probability of promoting anything, with its upper confidence bound below target. Under the complete null every promotion is false, so this equals the false discovery rate there.
   - **Mixed signal and noise:** the false discovery proportion among promotions.
   - **Power:** the rate at which planted signals of known strength are promoted.

   Passing establishes calibration against these model families, not universal error control.

3. **Single-agent research.** One agent proposing mechanism-backed hypotheses, compared against random or genetic search through the same gate. Zero survivors is an acceptable outcome; the success criterion is calibration plus an honest comparison with the baseline.

4. **Swarm and canonicalization.** Parallel agents sharing history, e-graph canonicalization, and effective-N sensitivity analysis. Repeat step 2 at this scale, since shared history adds new adaptive feedback.

5. **Data integration.** Add governance and social sources with immutable revisions, lineage, and per-source availability tests.

6. **Verified reference kernels.** Pre-register the five theorems of Section 6, have them independently reviewed, and freeze them. Build the Lean 4 and Bend 2 reference kernels and differential-test both against the engine. Exit with the pre-registered comparison, the winner adopted, and the loser deleted.

7. **Sources as a language feature.** Implement source declarations, binding, and certificates (Section 5), and move every existing feed onto them, deleting the per-feed code paths they replace. Exit when every existing feed is bound through a certificate and a new feed can be added without changing the kernel.

8. **Multi-asset data.** Build point-in-time universes and sources for equities, indices, commodities, and macro vintages; confirm which execution venues the operator may use and certify execution mappings for them; generalize the kernel from a single 24-hour perpetual to instrument kinds with their own calendars, settlement, and decision cadences; repeat the stage-0 feasibility check per asset class; and repeat the stage-2 calibration with null models matching the larger universe's cross-asset dependence, judged on a locked evaluation suite of null generators kept separate from those used to design the gate.

9. **Cross-domain exploration.** Implement the exploration, confirmation, and holdout periods of Section 10. Calibrate the whole two-stage procedure in simulation, including planted cross-domain signals, before any real campaign. Zero confirmed hypotheses is an acceptable outcome.

10. **Thesis track.** Implement thesis registration, retrieval logging, the fixed sizing rule, and scoring against the baselines of Section 12. This stage depends only on the ledger, so it starts as soon as stage 1 is complete and runs alongside the others. No skill claim is made before the pre-registered minimum number of resolved thesis clusters.

11. **Paper trading, then small live size.** A human decision, outside the autonomous scope. Compare live results with the gate's predictions. A large gap means the model failed somewhere: execution, costs, a regime change, parameter instability, decay of a real edge, or leakage.

Stages 0 and 2, and their repetitions in stage 8, matter most and are the easiest to skip. Before scaling the search, the system must show that its features can be reconstructed as they were available historically, and that its complete search and selection procedure meets its false-promotion targets under the specified market models. A correctly calibrated gate will still occasionally promote noise at its stated rate; the system is broken when the observed rate exceeds the target by more than sampling error. These checks cannot guarantee future profit, but they establish whether the research process meets its basic design requirements.

References

[1] Federal Reserve Bank of St. Louis. ALFRED: Archival Federal Reserve Economic Data. Online database.

[2] R. R. Bahadur and L. J. Savage. The nonexistence of certain statistical procedures in nonparametric problems. *Annals of Mathematical Statistics*, 1956.

[3] D. H. Bailey, J. Borwein, M. López de Prado, and Q. J. Zhu. The probability of backtest overfitting. *Journal of Computational Finance*, 2017.

[4] D. H. Bailey and M. López de Prado. The deflated Sharpe ratio: correcting for selection bias, backtest overfitting, and non-normality. *Journal of Portfolio Management*, 2014.

[5] Higher Order Company. Bend 2, version 2.0.37. `https://github.com/bendlang/bend`, 2026.

[6] Higher Order Company. Bend2 vs Lean. Project notes, `https://bend2.dev/notes/bend2-vs-lean/`, 2026.

[7] K. S. Brown, D. I. Spivak, and R. Wisnesky. Categorical data integration for computational science. *Computational Materials Science*, 2019.

[8] Y. Benjamini and Y. Hochberg. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 1995.

[9] G. W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 1950.

[10] D. R. Cox. A note on data-splitting for the evaluation of significance levels. *Biometrika*, 1975.

[11] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, 2021.

[12] C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold, and A. Roth. The reusable holdout: preserving validity in adaptive data analysis. *Science*, 2015.

[13] D. P. Foster and R. A. Stine. α-investing: a procedure for sequential control of expected false discoveries. *Journal of the Royal Statistical Society, Series B*, 2008.

[14] C. W. J. Granger and P. Newbold. Spurious regressions in econometrics. *Journal of Econometrics*, 1974.

[15] R. C. Grinold. The fundamental law of active management. *Journal of Portfolio Management*, 1989.

[16] D. Halawi, F. Zhang, Y.-H. Chen, and J. Steinhardt. Approaching human-level forecasting with language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[17] P. R. Hansen. A test for superior predictive ability. *Journal of Business and Economic Statistics*, 2005.

[18] C. R. Harvey, Y. Liu, and H. Zhu. . . . and the cross-section of expected returns. *Review of Financial Studies*, 2016.

[19] B. Hurst, Y. H. Ooi, and L. H. Pedersen. A century of evidence on trend-following investing. *Journal of Portfolio Management*, 2017.

[20] Hyperliquid. HIP-3: builder-deployed perpetuals. Protocol documentation, `https://hyperliquid.gitbook.io/hyperliquid-docs/hyperliquid-improvement-proposals-hips/hip-3-builder-deployed-perpetuals`.

[21] Z. Kakushadze. 101 formulaic alphas. *Wilmott*, 2016.

[22] E. Karger et al. ForecastBench: a dynamic benchmark of AI forecasting capabilities. In *International Conference on Learning Representations*, 2025.

[23] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 2009.

[24] Y. Li et al. R&D-Agent-Quant: a multi-agent framework for data-centric factors and model joint optimization. arXiv:2505.15155, 2025.

[25] A. W. Lo. The statistics of Sharpe ratios. *Financial Analysts Journal*, 2002.

[26] M. López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.

[27] QuantConnect. LEAN algorithmic trading engine. Open-source software, `https://github.com/QuantConnect/Lean`.

[28] T. J. Moskowitz, Y. H. Ooi, and L. H. Pedersen. Time series momentum. *Journal of Financial Economics*, 2012.

[29] J. P. Romano and M. Wolf. Stepwise multiple testing as formalized data snooping. *Econometrica*, 2005.

[30] D. I. Spivak. Functorial data migration. *Information and Computation*, 217:31–51, 2012.

[31] D. I. Spivak and R. E. Kent. Ologs: a categorical framework for knowledge representation. *PLoS ONE*, 2012.

[32] R. Sullivan, A. Timmermann, and H. White. Data-snooping, technical trading rule performance, and the bootstrap. *Journal of Finance*, 1999.

[33] P. E. Tetlock and D. Gardner. *Superforecasting: The Art and Science of Prediction*. Crown, 2015.

[34] S. Wang et al. Alpha-GPT: human-AI interactive alpha mining for quantitative investment. arXiv:2308.00016, 2023.

[35] J. Ville. *Étude critique de la notion de collectif*. Gauthier-Villars, 1939.

[36] I. Waudby-Smith and A. Ramdas. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society, Series B*, 2024.

[37] H. White. A reality check for data snooping. *Econometrica*, 2000.

[38] M. Willsey et al. egg: fast and extensible equality saturation. *Proceedings of the ACM on Programming Languages (POPL)*, 2021.